JAVA - TECHNICAL ASSESSMENT

Name:Movva Kishore
employee ID:362077

1) Once created, a String can't change. If you do str = "hello" then str = "world", you're pointing to a new String, not modifying the original.

How it helps with multithreading

1) No synchronization needed
   - Multiple threads can read the same String without locks.
   - Since it can't change, there's no risk of one thread modifying it while another reads it.

2) Safe sharing
   - Strings can be shared across threads without copying.
   - No race conditions because the value never changes.

2) Comparable:

- Used when an object knows how to compare itself
- Comparison logic is inside the class
- Method used: compareTo()

Real-time example of Comparable:

- A student compares himself with another student by marks

Comparator:

- Used when external logic compares objects
- Comparison logic is outside the class
- Method used: compare()

Real-time example of Comparator:

- A teacher decides how to compare students
  (by marks, by name, by age)
- Example use case:
  Sort students by name

3) When we call map.get(key), Java first calculates the hash code of the key using key.hashCode(). This hash value is converted into an index (bucket) in an internal array where the data is stored. Java then goes to that bucket.
In that bucket, Java checks each entry's key using equals() to find the exact match. If the key matches, it returns the value. If no key matches, it returns null.

4) • Hibernate requires a no-argument constructor because it creates objects automatically using reflection, without knowing what values to pass.

• Once the empty object is created, Hibernate sets the values from the database into the object's fields, so a no-arg constructor is needed.

5) Statement is used to execute simple SQL queries directly. The SQL query is sent to the database as it is, so it is less secure and slower because the database has to parse and compile the query every time.PreparedStatement is precompiled and uses placeholders , making it faster and safer . CallableStatement is used to call stored procedures in the database and is helpful when business logic is written inside the database itself.

6) @Transactional is used to make sure that a group of database operations run as one unit.
If all operations succeed, the changes are saved to the database.
If an exception occurs, Hibernate/Spring rolls back the transaction, meaning all changes are undone, so the database stays safe and consistent.

7) The equals() contract is important in Hibernate because Hibernate uses equals() to identify and compare entities in memory. If equals() is wrong, Hibernate may think two same records are different or two different records are the same.This can cause duplicate objects, wrong updates, or cache problems, so following the equals() contract keeps entity behavior correct and predictable.

8)Lazy fetching means data is loaded only when you use it, while eager fetching means data is loaded immediately along with the main object.Lazy fails when you access lazy data after the session is closed .

9)The default isolation level in most databases is READ COMMITTED.
It means a transaction can only read data that has already been committed, not uncommitted data.

10)JDBC resources should be closed to free database connections, memory, and locks.Using finally or try-with-resources ensures they are closed even if an exception occurs, preventing resource leaks.

11) 1) == checks whether two object references point to the same memory location (same object).

2) .equals() checks whether two objects have the same content or values (logical equality).

12) The N+1 problem in Hibernate happens when Hibernate runs one query to load the main data (1) and then separate queries for each related record (N).

This causes many unnecessary database calls, making the application slow.

13) In Java, the transient keyword means the variable will not be saved during serialization. When the object is converted to a stream, that field is ignored.

In Hibernate, transient means the field is not mapped to the database. Hibernate will not store or fetch that field from the table.

15) POST data is NOT carried to the redirected page. A new request is created.

What HTTP status code is used?

Usually 302 (Found) or 303 (See Other).

Browser vs Server behavior:

Redirect → Browser makes a new request

Forward → Server internally moves request (same data)

Flow:

Browser sends POST

Server responds with redirect (302/303)

Browser sends a new GET request

POST data is gone

16) Hibernate/JPA cannot map Optional properly to database columns.

Optional is meant for method return values, not for fields.

Using it as a field can cause mapping errors and unexpected behavior.

17) Hibernate uses Dirty Checking.

Internal data structure:

Hibernate keeps a snapshot (copy) of entity state in the Persistence Context (first-level cache).

When is SQL fired?

SQL UPDATE is fired:

At transaction commit Or during flush

If entity is detached:

Hibernate does not track changes → no update happens unless you reattach (merge).

18) To reduce boilerplate code and speed up development.

Spring Boot auto-configures things based on:

Classpath

Dependencies

Defaults

You write less config, focus more on business logic.

20)

ClassNotFoundException

Occurs at runtime

- JVM cannot find class when loading dynamically
- Checked exception
- NoClassDefFoundError

Class was present at compile time

- Missing at runtime
- Error (not exception)

Not found while loading → Exception

Found before, missing now → Error

21) String s1 = "Java";
String s2 = new String("Java");

System.out.println(s1 == s2) => false=> == compares memory reference → different objects
System.out.println(s1.equals(s2))=> true=> equals() compares content → same text

22) You are modifying the list while looping using enhanced for loop.This is not allowed.

23) static block runs first, before main().

24) SELECT query must use executeQuery(), not executeUpdate().

25) Entity must have a no-argument constructor.Hibernate cannot create the object.

26) Division by zero throws exception → caught
finally always executes

27) Spring cannot find a UserService bean to inject.

28) If name parameter is not sent in the request,
getParameter("name") returns null, and calling toUpperCase() on null crashes.

30) Browser sends HTTP request
DispatcherServlet receives it
Maps request to Controller
Controller calls Service
Service calls Repository
Repository executes DB query (Hibernate/JPA)
Data flows back → Controller → Response to browser

31)
Enable SQL logs to see actual query
Fix N+1 problem (use fetch join)
Add proper indexes in DB
Avoid fetching unnecessary columns
Use pagination
Enable second-level cache if needed

32)
A – Atomicity:
Money debit and credit happen together or not at all

C – Consistency:
Total money before and after transfer remains correct

I – Isolation:
Two users transferring money don’t affect each other

D – Durability:
Once transaction succeeds, data is saved even after crash

33) JSP
Server-side rendering
Tightly coupled with backend
Older technology
Thymeleaf
Server-side rendering
Better Spring Boot integration
Clean HTML templates
React
Frontend only
Backend exposes REST APIs
Loose coupling (best scalability)

34) Centralized error handling for all controllers

35)

Reuse database connections instead of creating new ones every time
Why important:

- Creating DB connections is slow
- Improves performance
- Handles more users efficiently
- Spring Boot uses HikariCP by default.

37)

```
 public User getUserByEmail(String email) throws Exception {

String sql = "SELECT id, name, email FROM users WHERE email = ?";
PreparedStatement ps = connection.prepareStatement(sql);
ps.setString(1, email);

ResultSet rs = ps.executeQuery();

if (rs.next()) {
User user = new User();
user.setId(rs.getInt("id"));
user.setName(rs.getString("name"));
user.setEmail(rs.getString("email"));
return user;
}
return null;
}
```

38) One User can have many Orders.

39)

```
protected void doPost(HttpServletRequest req, HttpServletResponse res)
throws IOException {

String username = req.getParameter("username");

HttpSession session = req.getSession();
session.setAttribute("user", username);
```

res.sendRedirect("dashboard.jsp");
}

40) ${name}
Cleaner, readable, no Java code in JSP.

41) try (Connection con = getConnection();
PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery()) {
// process data
}
Automatically closes DB resources.

42)
for(let i=0;i<3;i++){}
console.log(i);

43)
Event bubbling:
Event moves from child → parent.
Used to prevent parent event from firing.

44)
Reason:
var is hoisted.

46) orders is LAZY and accessed after session is closed.
SELECT u FROM User u JOIN FETCH u.orders

47). String sql = "SELECT * FROM users WHERE email = ?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, email);
ResultSet rs = ps.executeQuery();

49) Why:
Spring can't find a bean to inject.

How Spring solves it normally:
Spring Data JPA auto-generates implementation for repository interfaces.

50) POST form

Save data

Redirect to GET page

Refresh repeats GET, not POST

51) User u = new User();

u → Stack

new User() → Heap

Stack: method calls, local variables

Heap: objects, shared data

52) Because multiple threads can modify structure simultaneously.

Make it thread-safe:

- Collections.synchronizedMap()
- ConcurrentHashMap

53)

Creates internal array of size 10

No elements added yet

Avoids resizing later

54)

Rule:

Equal objects must have same hashCode

Otherwise data lookup fails.

55) final → cannot change (variable/method/class)

finally → always executes

finalize() → GC cleanup (deprecated)

56)void test(int a)

void test(Integer a)

This is overloading, decided at compile time.

Overriding is decided at runtime.

58) No,

Static methods belong to class, not object.

This is method hiding, not overriding.

59) To avoid Diamond Problem
Interfaces solve this safely.

60) throw → actually throws exception
throws → declares possible exception

61)
int a = 10;
Integer b = a;
Performance issue:
Creates unnecessary objects → slower in loops.

62)
ArrayList: dynamic array (fast read)
LinkedList: doubly linked list (fast insert/delete)

63) JVM cannot access it
→ Runtime error: main method not found

64) No,Interfaces cannot create objects.

65)Interface with no methods.
Used to signal JVM.

66)
Creation
Use
Eligible for GC
Garbage collected

67) Class loading fails
→ ExceptionInInitializerError

68)
JVM: runs bytecode
JRE: JVM + libraries
JDK: JRE + compiler/tools

69) Because wait releases lock, and lock is required to call it.

70) notify() → wakes one thread
notifyAll() → wakes all waiting threads

71) Yes
clone()
reflection
deserialization

72) Original exception is lost
finally exception wins

73) Saves memory
Faster comparisons
Reuses objects

74) Shallow: copies reference
Deep: copies object + internal objects

75) Serialization → convert object to byte stream
serialVersionUID ensures version compatibility

76) Reflection lets you inspect/change classes and methods at runtime.
Risky because it breaks encapsulation, is slower, and can cause security issues.

77) Checked: compile-time (IOException)
Unchecked: runtime (NullPointerException)

78) Yes. Static methods belong to the class, not objects.

79) If method isn't synchronized → race condition and wrong data.

80) Immutable objects can't change → thread-safe without synchronization.

81) Load driver
Get connection
Create statement
Execute query

Process ResultSet
Close resources

82) Precompiled
Reused
Prevents SQL injection

83) Memory leak + DB connection exhaustion → app crash.

84) Reuse DB connections.
Required for performance and scalability in production.

85)executeQuery() → SELECT
executeUpdate() → INSERT/UPDATE/DELETE
execute() → unknown SQL

86) Using PreparedStatement separates SQL from data.

87) Move forward/backward in ResultSet.
DB changes don’t affect it.

88) Changes are not saved → rolled back.

89) Normal: one query at a time
Batch: many queries together (faster)

90) Security risk + hard to change + exposed in code.

91) Use con.rollback() inside catch block.

92) ClassNotFoundException → no DB connection.

93) Yes, using multiple connections.

94) Connection factory with pooling support.

95) Enable logging in JDBC framework / DB driver.

96) save → returns ID
persist → no return
saveOrUpdate → insert or update

97) Session-level cache. Always enabled.

98) First: Session scope
Second: SessionFactory scope (shared)

99) Hibernate auto-detects entity changes at commit/flush.

101) Hibernate ignores the class → no mapping.

102) Generates equals/hashCode → breaks Hibernate proxies.

103) Cascade propagates operations.
ALL = save, update, delete cascade.

104) Extra join table or wrong relationship.

105) OneToOne → one record
ManyToOne → many records point to one

106) Deletes child when removed from parent.

107) IdClass → separate key class
EmbeddedId → embedded object

109) Use setFirstResult() and setMaxResults().

110) get → null if not found
load → proxy, exception later

111) Marks business key (like email).

112) Infinite loops + performance issues.

113) Constraint violation or shared identity mapping.

114) Optimistic → version check
Pessimistic → DB lock

115) Used for optimistic locking.

116) Memory + connection leak.

117) spring.jpa.show-sql=true

119) Loads unnecessary data → slow performance.

120) @Table → table name
@Column → column name

121) init → service → destroy

122) GET → visible, cacheable
POST → secure, not cached

123) Runs servlets (e.g., Tomcat).

124) Cookies / URL rewriting.

126) Session invalidated → user logged out.

127) Redirect → new request
Forward → same request

128) Breaks MVC, hard to maintain.

129) request, response, session, application, out

131) Escape output (<c:out>).

132) Session data is shared.

133) session.invalidate()

134) Intercepts requests (auth, logging).

135) Listens to lifecycle events.

136) Auto-config, embedded server, less setup.

137) Spring injects Repository into Service.

139) Same behavior, different semantics.

140) Based on classpath + properties.

141) Central configuration file.

142) Use @ControllerAdvice.

145) Create → inject → init → use → destroy

146) id → unique
class → reusable

147) Content + padding + border + margin

148) none → removed
hidden → space kept

149) Start
End
Promise
Timeout
Sync → microtask (Promise) → macrotask (setTimeout)

150) Event flows from child to parent.